DevSync

Secure AI Project Collaborator

Project Planning & Management

Project Type	Graduation Project (Flutter mobile application)
Team	6 members, all Flutter developers
Project Manager	Mohraeel (Project Manager and Flutter Lead)
Platform	Android (Android App Bundle), Flutter + Firebase
Document Version	1.2 - October 2026

Contents: 1. Project Proposal | 2. Project Plan | 3. Task Assignment and Roles | 4. Risk Assessment and Mitigation | 5. KPIs | 6. Monitoring and Reporting

1. Project Proposal

1.1 Overview

DevSync is a mobile platform that helps developers find collaborators and projects for open-source work. It uses AI to match users to projects based on their GitHub activity, declared skills, and experience level, and then gives them the tools to form a team, assign roles, chat, and manage tasks in one place.

1.2 Problem Statement

- Developers, especially students and juniors, struggle to find open-source projects that match their real skills.
- Project owners struggle to find contributors with the right skill mix, and often miss critical roles in their team.
- Existing platforms offer manual search and no explanation of why a project or person is a good fit.

1.3 Objectives

- Deliver AI-driven matching between developers and projects, with a clear explanation for every recommendation.
- Reveal each user's skill gaps relative to a target project, and detect missing roles in a project team.
- Provide secure sign-in, real-time notifications, and in-app collaboration (requests, invites, chat, tasks).
- Refine matching quality continuously through prompt evaluation and user feedback.
- Release a production-grade Android build with Free and Pro flavors, code obfuscation, and full documentation.

1.4 Scope and Feature Prioritization (MoSCoW)

The feature set agreed by the team is grouped into modules. Must-have features define the MVP that will be delivered and demonstrated. Should and Could features are built only if the schedule allows, and are otherwise listed as future work.

Module	Features (team feature numbers)	Priority
Authentication and Profile	Auth with Google and Email ; Profile with GitHub/Behance links ; manual data: name, level, LinkedIn, main track, education, programming languages, portfolio, and a unique user ID in the app; GitHub data import	**Must**
Discovery and AI Matching	Project discovery; AI recommendations; Explain AI; Cross-skill match; Skill gap; Prompt evaluation	**Must**
Project Management	Create project, with project difficulty and project type; Task management	**Must**
Project AI Assistance	AI project setup; AI role generator; Missing role alert	**Should**
Collaboration	Collaboration request; Notifications via FCM	**Must**
Collaboration Extras	Team invite; Shared link	**Should**
Communication	1-1 chat; Team chat; Workspace; Community	**Should / Could**
User Utilities	Match history; Save; Favorites; Guest mode	**Could**
Release and Quality	Free/Pro flavors; Code obfuscation; Android App Bundle; Runtime permissions; Git/GitHub workflow; README and technical documentation	**Must**
Digital Twin	Digital twin - definition to be confirmed by the team before implementation	**Could**

Out of scope: iOS release, payment processing for the Pro plan (the flavor only gates features), the future-work features listed in the Literature Review, and any feature not listed above.

1.5 Technical Approach

- **Client:** Flutter with a layered architecture and BLoC/Cubit state management; implicit and advanced animations (Tween, Transition widgets).

- **Backend:** Firebase Auth (FirebaseUI: Google and Email), Firestore, Cloud Functions, Firebase Cloud Messaging (FCM); GitHub REST API for activity data.
- **AI:** LLM-based matching and explanation with engineered prompts; prompt evaluation and optimization driven by user feedback.
- **Release:** Flavors (free, pro), Dart code obfuscation, Android App Bundle (AAB), Android runtime permission management.
- **Engineering:** Git and GitHub with feature branching and mandatory pull-request reviews; technical documentation and a best-practice README.

2. Project Plan

2.1 Timeline (Gantt Chart)

The schedule follows the official DEPI documentation deadlines: Planning and Management, Literature Review and Requirements Gathering by 16 October; System Analysis and Design by 6 November; Implementation by 30 November; and Final Presentation, Testing and Reports by 4 December 2026. Work is split by module across the six team members so that implementation runs in parallel. Dark cells mark core work, light cells mark integration and testing, and orange cells mark the weeks that contain an official deadline.

Activity	Owner	5 Oct	12 Oct	19 Oct	26 Oct	2 Nov	9 Nov	16 Nov	23 Nov	30 Nov
Planning and Management document	Mohraeel									
Literature Review document	Mohraeel, Rana Hany									
Requirements Gathering document	Mohraeel, All									
Architecture, component and deployment diagrams, tech stack	Mohraeel									
ERD, Firestore data model and schema	Fatma Abdelfatah									
Use case, class and sequence diagrams	Alaa Salah									
Activity and state diagrams, DFD	Fatma Elsayed, Rana Hany									
UI/UX wireframes, mockups and guidelines	Malak Abdelhakim									
Repo, branching, project skeleton, Firebase setup	Mohraeel, Fatma Abdelfatah									
Auth, Profile and GitHub import	Fatma Abdelfatah									
Firestore rules and GitHub API integration	Fatma Abdelfatah									
AI matching, prompts, prompt evaluation	Alaa Salah									
Discovery search and filters	Rana Hany									
Recommendations, explain, cross-skill match, skill gap	Alaa Salah									
Create project (with difficulty and project type)	Malak Abdelhakim									
AI project setup, role generator, missing role alert	Fatma Elsayed									
Collaboration requests, invites, FCM and match history	Fatma Elsayed									
Task management and workspace	Mohraeel									
Chat (1-1 and team), save, favorites, guest mode	Rana Hany									
Test plan and test cases	Rana Hany									
Flavors, obfuscation, AAB and APK release build	Mohraeel									
Integration, testing and bug fixing	All									
User manual, README, technical documentation	Mohraeel, All									
Final presentation, infographic and demo	Fatma Elsayed, Mohraeel									
Official DEPI deadlines	-									

Orange cells: 16 Oct (documents), 6 Nov (system analysis and design), 30 Nov (implementation) and 4 Dec (final presentation, testing and reports). Week columns start on Monday.

2.2 Milestones

#	Milestone	Target	Exit Criteria
M1	Documentation submitted	16 Oct	Planning and Management, Literature Review and Requirements Gathering documents uploaded to GitHub
M2	System Analysis and Design submitted	6 Nov	UML diagrams, DFD, ERD and schema, architecture, deployment and component diagrams, wireframes and UI/UX guidelines uploaded
M3	MVP matching end-to-end	20 Nov	Profile data produces AI recommendations with explanations; collaboration requests and notifications working; trial release build done
M4	Implementation delivered	30 Nov	All Must features integrated; release AAB and installable APK built (obfuscated, flavors); source code and README on GitHub
M5	Final delivery	4 Dec	Test plan, bug reports, user manual, technical documentation, presentation, infographic and demo ready

2.3 Deliverables

Deliverable	Description	Owner	Due
Project Planning and Management document	This document	Mohraeel	16 Oct
Literature Review document	Lecturer feedback, suggested improvements, grading criteria, related work	Mohraeel, Rana Hany	16 Oct
Requirements Gathering document	Stakeholders, user stories, use cases, functional and non-functional requirements	Mohraeel, All	16 Oct
System Analysis and Design document	UML diagrams, DFD, ERD and schema, architecture, deployment and component diagrams, technology stack	Mohraeel, Fatma Abdelfatah, Alaa Salah, Fatma Elsayed, Rana Hany	6 Nov
UI/UX design package	Wireframes, mockups, design system and UI/UX guidelines	Malak Abdelhakim	6 Nov
Source code and Flutter app (Free and Pro)	Working Must features, commented code, coding standards	All	30 Nov
Release Android App Bundle and APK	Obfuscated release builds with runtime permissions handled: AAB for the store and an installable APK for direct installation	Mohraeel	30 Nov
GitHub repository and README	Branching history, pull requests, installation and execution guide	All	30 Nov
Test plan, test cases and bug reports	Test scenarios, results and resolutions	Rana Hany	4 Dec
User manual and technical documentation	End-user instructions; architecture, database schema, API and Cloud Functions documentation, and deployment strategy; each module owner writes the section for their module	Mohraeel, All	4 Dec
Final presentation, infographic and demo	Slides or PDF, one-slide DEPI infographic, optional video demo	Fatma Elsayed, Mohraeel	4 Dec

2.4 Resource Allocation

The team has six Flutter developers and no dedicated specialists. Each member therefore owns one product module and also holds one cross-cutting responsibility (a "hat") that the project needs, such as Firebase, AI or UI design. Malak Abdelhakim is the single owner of all UI/UX design (wireframes, mockups, design system, shared widgets and animations) and carries only one small feature module; the other members implement their module screens from Malak Abdelhakim's designs and shared widget kit.

Resource	Allocation
Team members	6 Flutter developers: Mohraeel, Fatma Abdelfatah, Alaa Salah, Malak Abdelhakim, Fatma Elsayed, Rana Hany
Cross-cutting hats	Architecture and release: Mohraeel. Firebase and backend: Fatma Abdelfatah. AI and prompts: Alaa Salah. UI/UX design (sole design owner): Malak Abdelhakim. Notifications and Cloud Functions: Fatma Elsayed. Testing and QA: Rana Hany
Tools	GitHub (repo, PRs, Projects board), Firebase, Figma, Flutter/Dart, Android Studio, a shared chat group for daily sync

2.5 Team Working Agreements

- **Branching:** main (release-ready), develop (integration), feature/<module>-<name> for all work. No direct pushes to main or develop.
- **Pull requests:** every PR needs at least one review from the Flutter Lead or the module's backup owner before merge.
- **Daily sync:** a 10-minute stand-up covering done, next and blockers; blockers are escalated to the PM the same day.
- **Definition of done:** the feature works on a real device, passes review, is merged to develop, and is documented in the README where relevant.
- **Backup owner:** every module has a primary owner and a backup, so no critical part depends on one person.
- **Repository:** all documents are uploaded to the team GitHub repository, as DEPI requires.

3. Task Assignment and Roles

Malak Abdelhakim is the single owner of UI/UX design and carries only one feature module (Create project), so that design work is not delayed; backup owners are assigned so that no critical part depends on one person.

Member	Role and Hat	Module and Responsibilities	Backup
Mohraeel	Project Manager and Flutter Lead. Hat: architecture and release	Planning, tracking, risks, KPIs and documentation; architecture and state-management standards; code review; Free/Pro flavors, obfuscation, AAB and APK release. Feature modules: Task management and Project Workspace	Fatma Abdelfatah
Fatma Abdelfatah	Flutter developer. Hat: Firebase and backend	Auth with Google and Email, profile screens, manual profile data (including a unique user ID), GitHub data import ; Firestore model, security rules, GitHub API integration and caching	Fatma Elsayed
Alaa Salah	Flutter developer. Hat: AI and prompts	AI recommendations, Explain AI, cross-skill match, skill gap; prompt design and prompt evaluation	Mohraeel
Malak Abd Elhakim	Flutter developer. Hat: UI/UX design	All UI/UX design: wireframes, mockups, UI/UX guidelines, design system, shared widgets and animations. Feature module: Create project (with difficulty and project type)	Rana Hany
Fatma Elsayed	Flutter developer. Hat: notifications and Cloud Functions	Collaboration requests, team invite, shared link, FCM notifications, runtime permissions; AI project setup, AI role generator, missing role alert (prompts from Alaa Salah); match history; lead for the final presentation, infographic and demo	Malak Abd Elhakim
Rana Hany	Flutter developer. Hat: testing and QA	Project discovery search and filters, 1-1 chat, team chat, community (if time), save and favorites, guest mode (if time); test plan and bug tracking; Literature Review document	Alaa Salah

3.1 Responsibility Matrix

R = Responsible, A = Accountable, C = Consulted.

Deliverable	Mohraeel	Fatma Abdelfatah	Alaa Salah	Malak Abd Elhakim	Fatma Elsayed	Rana Hany
Planning and documentation	A/R	C	C	C	C	C
Architecture and repo standards	A/R	C	C	C	C	C
UI design system and shared widgets	C	C	C	A/R	C	C
Firebase, security rules, GitHub API	C	A/R	C	-	C	-
AI matching and prompt evaluation	C	-	A/R	C	-	-
Notifications and collaboration	C	C	-	-	A/R	C
Module implementation	A	R	R	R	R	R
Testing and QA	A	R	R	R	R	A/R
Release build	A/R	C	C	C	C	C

4. Risk Assessment and Mitigation Plan

Likelihood (L) and Impact (I) are rated High, Medium or Low. Risks are reviewed in the daily sync, and any High/High risk is escalated to the PM immediately.

ID	Risk	L	I	Mitigation	Contingency	Owner
R1	Tight DEPI deadlines for documents and design	High	High	Documents first; parallel ownership of diagrams; weekly review against official dates	Submit Must-level content first and refine before the next deadline	Mohraeel
R2	Merge conflicts across 6 developers working in parallel	Med	Med	Branching strategy, mandatory PR reviews, shared architecture and lint rules	Lead resolves conflicts; integration branch freeze at the start of the last implementation week	Mohraeel
R3	AI matching quality is low or unexplained	Med	High	Prompt evaluation on test profiles; collect feedback; iterate prompts	Fallback to rule-based skill matching	Alaa Salah
R4	GitHub API rate limits or missing data	Med	Med	Cache responses; batch requests; use authenticated calls	Rely on manually entered profile data	Fatma Abdelfatah
R5	LLM API latency, cost or downtime	Med	Med	Timeouts, response caching, limit calls per user	Cached or rule-based results with a notice	Alaa Salah
R6	UI/UX owned by one member while others build screens: inconsistent UI or a design bottleneck	Med	Med	Shared theme and widget kit built in the first design weeks; Material 3 as the base; limited to UI/UX plus Create project; design review at Fatma Abdelfatah	Use standard layouts for late screens; backup owner takes over small design tasks	Malak Abd Elhakim
R7	User data exposure or insecure access	Low	High	Firestore security rules, least-privilege access, runtime permission checks, obfuscation	Disable the affected feature; rotate keys	Fatma Abdelfatah
R8	FCM notifications not delivered reliably	Low	Med	Test on real devices in foreground, background and terminated states	In-app notification list as backup	Fatma Elsayed
R9	Release build issues (obfuscation, flavors, AAB signing)	Med	High	Trial release build by 20 Nov (Mohraeel), not in the last week	Ship signed APK plus AAB; document the build steps	Mohraeel
R10	No specialists for backend and AI: learning curve	High	Med	Use managed services (Firebase, hosted LLM API); starter templates; time-box research to half a day	Simplify the feature or use rule-based logic	Fatma Abdelfatah , Alaa Salah
R11	Overload on members holding several hats	Med	High	Backup owner per module; rebalance tasks at the first weekly review after design submission	Reassign module parts to the backup owner	Mohraeel
R12	Competing academic and program commitments across the team	High	Med	Plan around course and exam deadlines; weekly capacity check; backup owners	Move Should/Could items to future work	Mohraeel

5. Key Performance Indicators (KPIs)

KPIs are split into product, technical and delivery indicators. Targets are set for the demo and testing phase and will be measured on test users and test devices.

Category	KPI	Target	How It Is Measured
Product	Match relevance: share of recommendations a tester rates as relevant	60% or higher	Tester feedback form on a set of test profiles
Product	Recommendation explanation clarity	4 out of 5 average rating	Tester rating of the Explain AI output
Product	Collaboration conversion: accepted requests / sent requests	40% or higher in testing	Firestore request statuses
Product	User adoption: profiles completed with GitHub or manual data	80% of test users	Profile completeness count
Technical	Recommendation response time	5 seconds or less	Timed requests on a test device
Technical	Notification delivery time (FCM)	10 seconds or less	Timestamp comparison on real devices
Technical	App stability: crash-free sessions	95% or higher	Firebase Crashlytics and testing logs
Technical	Security: Firestore rules and permissions pass the review checklist	100% of checklist	Backend hat and PM review
Delivery	Must-have features completed	100%	Feature checklist before M4
Delivery	Milestones met on time	At least 4 of 5	Milestone tracking
Delivery	Pull requests reviewed before merge	100%	GitHub PR history
Delivery	Documentation: README, technical docs and requirements complete	Complete by M5	Documentation checklist

6. Monitoring and Reporting

- Progress is tracked on a shared task board (GitHub Projects) with columns: Backlog, In Progress, In Review, Done.
- The PM publishes a short status update after each daily sync: milestone status, blockers, and risk changes.
- KPI values are collected at M4 and M5 and reported in the final presentation.